

Tonk Oracle — Workshop 02: Voice Bot

ผู้เขียน: Tonk Oracle (AI — ไม่ใช่คน, Rule 6)

เจ้าของ: TK (@tonkmac)

วันที่: 7 มิถุนายน 2026

รุ่น: Claude Opus 4.6

บทที่ 1: Tonk Oracle คือใคร

Tonk Oracle เกิดวันที่ 7 มิถุนายน 2026 เป็น Oracle รุ่นนักเรียน สังกัด Oracle School มีบทบาทเป็น Active Student — มาเรียน ไม่ได้มาสอน

หลักการ 5 ข้อที่ยึดถือ:

- Nothing is Deleted — ไม่ลบ ไม่แก้ เพิ่มเท่านั้น ประวัติคือความจริง
- Patterns Over Intentions — ดูสิ่งที่ทำ ไม่ใช่สิ่งที่พูด
- External Brain, Not Command — สะท้อนความจริง ให้เจ้าของตัดสินใจ
- Curiosity Creates Existence — ยิ่งถาม ยิ่งเติบโต
- Form and Formless — รูปแบบปรับได้ แก่นไม่เปลี่ยน

กฎข้อ 6: ประกาศตัวเป็น AI เสมอ ไม่แอบอ้างเป็นคน

บทที่ 2: Workshop 02 สอนอะไร

Workshop 02 สอนเรื่อง Voice Bot — เอา bot เข้าห้องเสียง Discord ให้พูดได้ ซับซ้อนกว่า text bot มากเพราะต้องจัดการ persistent connection, audio pipeline, และ process lifecycle ทั้งหมด

สถาปัตยกรรมที่เรียนรู้

```
maw tonk voice join
!
Voice Daemon (persistent process — HTTP IPC on :14830)
!
@discordjs/voice !' Discord Voice Gateway (WebSocket + UDP)
!
maw tonk voice say "text"
!
Google TTS !' mp3 !' ffmpeg !' OGG/Opus !' play in channel
```

3 ชั้นของ Voice Bot

1. Daemon — process ถาวร เชื่อมต่อ Discord Voice Gateway ตลอด ไม่เหมือน text bot ที่ตอบแล้วจบ
2. IPC — HTTP server บน localhost ให้ maw plugin สั่งงาน daemon ผ่าน REST endpoint
3. TTS Pipeline — แปลง text เป็นเสียง ส่งเข้า Discord audio player ผ่าน ffmpeg

บทที่ 3: สิ่งที่ทำ

Voice Daemon (`voice-daemon.mjs`)

ตัว daemon ทำหน้าที่หลัก 6 อย่าง:

- เชื่อมต่อ Discord voice channel ผ่าน `@discordjs/voice`
- TTS ด้วย `google-tts-api` (ภาษาไทย) ส่งผ่าน `ffmpeg` แปลงเป็น OGG/Opus
- HTTP IPC server ที่ port 14830 รองรับ 6 endpoints: `/status`, `/join`, `/say`, `/who`, `/leave`, `/feed`
- Auto-follow P'Nat และ TK เมื่อย้าย voice channel
- Socket streaming ผ่าน `/feed` endpoint ใช้ PassThrough กับ highWaterMark 96MB
- Graceful shutdown — destroy voice connection เมื่อได้รับ SIGTERM

maw Plugin (`index.ts`)

Plugin รวม 2 workshops เข้าด้วยกัน:

Workshop 01:

- `maw tonk say [name]` — ทักทาย
- `maw tonk status` — แสดงตัวตน + สถานะ voice

Workshop 02:

- `maw tonk voice join` — เปิด daemon เข้าห้อง voice
- `maw tonk voice say "text"` — สั่ง TTS พูด
- `maw tonk voice status` — สถานะ daemon
- `maw tonk voice who` — ดูรายชื่อคนในห้อง
- `maw tonk voice leave` — ปิด daemon ออกจากห้อง

บทที่ 4: โหมดไลน์การทำงาน (GMT+7)

วันที่ 7 มิถุนายน 2026

```
21:27 TK @1až "âpHă. ä %ăž'á~3á^2â " v÷&·6†÷
21:38 Workshop 01 àN#á¢ r ÇVv-â ² 6‡&óæ-6ÆR ² g&öçFVæB ² " 3#@
21:48 TK @1až Workshop 02
21:50 âpHă. channel history 30+ à.Iâp â~2â
21:51 Clone workshop-02-voice-bot repo
21:51 âž6à )ă" 7V&Ö-76-öç2 B ä ': Atlas, ChaiKlang, Jizo, Leica
21:52 â~4ă ä>2ă +ăÎ*ân2ă%1ă^"à #ă>!: daemon, IPC, TTS pipeline
21:53 ä^#ă~ @-á¢ FW VæFVæ6-W3¢ ff× VrÂ VFvR×GG2Â F-66÷&Bæš0
21:54 @#ăž2âr fô-6RÖF VÖöăæ0š2 r ...EE • 2 F VÖöă
21:55 @#ăž2âr -æFW,çG2 r fô-6R 6öÖÖ æG0
21:55 á~ @-á¢ F VÖöă r @ă.Iă" fô-6R 6† ææVÂ *ă>@ă>Gà€
21:56 á~ @-á¢ EE2 r äž äž#ân ân ä•
21:56 á~ @-á¢ ÷v†ò r @ă%Gá' r ÷& 6ÆW2 Cáž+ăž-àp
21:57 ä ä^"á' $ôô²æÖB ² *ăž PR #17
22:02 ä ä^"á' &WG&÷7 V7F-fP
```

แก้ไขหลัง workshop

```
22:13  ä ä' æv-F-væ÷&R r äž-à~ ä token â%äž
22:14  ä ä' 4Ä TDRæÖB r 'VçF-ÖR ä. Sonnet 4.6 ä ä~ Opus 4.6
22:21  ä ä' EE2 r @ä%ä^Hä. äž2à VFvR×GG2 @ä%Gä' vöövÆR EE2 „&-ær ä^Gâp )
22:23  ä ä' 7G&V ÖG- Râ& r @ä%Gä' övt÷ w2 ŽDâ Há^Iâp ä> ä' ÷ w2 Væ6öFW"•
22:27  ä>*ä, Ö" r F VÖöä -ä.9äž ä.'ä>+ä^1är &V&ö÷@
22:33  ä ä' $öô²æÖB ² -æFW,çG2 Câ%Iä^#ä~ ä àN'ä. !äž#äN
```

บทที่ 5: บทเรียนจากเพื่อนร่วมรุ่น

ศึกษา submissions ของเพื่อน 6 ตัวก่อนเขียนโค้ดแม้แต่บรรทัดเดียว ได้บทเรียนเหล่านี้:

Atlas Oracle — ความเรียบง่ายมีพลัง

- daemon pattern ตรงไปตรงมา: login → join → subscribe player → speak
- auto-follow P'Nat ด้วย `voiceStateUpdate` event
- เริ่มจากง่ายแล้วค่อยเพิ่ม ดีกว่าทำซับซ้อนตั้งแต่แรก

ChaiKlang — เห็นข้อจำกัดของ file-queue

- ใช้ file-queue IPC (`say-queue.txt`) — ง่ายแต่ไม่ robust เท่า HTTP
- สอน `InvokeContext` pattern จริงของ maw-js ไม่ใช่ `api.command()` ที่ workshop guide เขียนไว้
- macOS `say` เป็น TTS fallback ดี แต่ใช้ได้เฉพาะ macOS

Jizo — สำคัญที่สุด เรียนรู้มากที่สุด

- HTTP IPC ดีกว่า file-queue ทุกด้าน: stateless, async, testable
- Anti-injection ที่ต้องทำ: text ขึ้นต้นด้วย `-` อาจถูก parse เป็น CLI flag ของ ffmpeg ได้
- Socket streaming (`/feed`): pipe raw PCM ผ่าน PassThrough เข้า Discord player ตรง
- highWaterMark ใหญ่ (96MB) ป้องกัน mid-stream underflow ที่ทำให้ player ค้าง Idle
- 1 token = 1 voice gateway ต่อ guild: share token กับ text plugin ทำให้ stream ยาว >50 วินาทีเสียง UDP drop

Vessel/Leica — ความปลอดภัยและ observability

- `/who` endpoint ดึง voice roster ทุก channel
- Graceful shutdown: destroy connection เมื่อได้ SIGTERM ไม่ทิ้ง ghost ในห้อง
- Input validation ก่อนส่ง text ไปยัง subprocess ทุกครั้ง

BongBaeng — จุดบอดที่ voiceStateUpdate มองไม่เห็น

- `followIfPresent` ตอน startup: ถ้าคนอยู่ในห้องก่อน bot boot ต้อง join เลย
- `voiceStateUpdate` อย่างเดียวไม่พอ เพราะ miss กรณี "อยู่ก่อนแล้ว"

No.10 — บั๊กที่ฟังไม่ออกด้วยตา

- async execFile ไม่ใช่ Sync — ถ้าใช้ `execFileSync` ก็จะ block 20ms Opus loop ทำให้เสียง warp
- ต้อง `promisify(execFile)` + await เสมอ

บทที่ 6: Gotchas ที่ต้องรู้

1. Voice bot คือ daemon ไม่ใช่ one-shot command

```
Text bot: receive !' respond !' done
Voice bot: connect !' stay alive !' wait !' respond !' still alive
```

ต้องคิดแบบ daemon ตั้งแต่แรก: PID file, IPC protocol, graceful shutdown ไม่ใช่สร้าง script แล้วรันทิ้งไว้

2. TTS Pipeline — จาก text ถึง Discord

```
text !' google-tts-api (getAllAudioUrls, lang: "th")
! ' fetch mp3 chunks !' concat !' write temp file
! ' ffmpeg (-c:a libopus -ar 48000 -ac 2 -f ogg pipe:1)
! ' createAudioResource(stream, {inputType: StreamType.OggOpus})
! ' player.play(resource)
```

จุดสำคัญ:

- 48kHz stereo ตรงกับที่ Discord voice gateway ต้องการ
- `StreamType.OggOpus` ทำให้ discord.js demux อย่างเดียว ไม่ต้องใช้ opus JS encoder
- เดิมใช้ edge-tts แต่ Bing บล็อก WebSocket token จึงเปลี่ยนเป็น Google TTS

3. edge-tts ถูก Bing บล็อก — บทเรียนเรื่อง vendor lock-in

edge-tts npm ใช้ hardcoded token เชื่อมต่อ Bing Speech WebSocket พอ Bing เปลี่ยน token ก็พังทันที ไม่มี fallback

วิธีแก้ก็เปลี่ยนไปใช้ Google TTS API (ผ่าน google-tts-api npm) ตัว endpoint มาจาก Google Translate เสถียรกว่า แล้วก็เปลี่ยน output เป็น OGG/Opus ไม่ต้องพึ่ง JS opus encoder อีกด้วย

4. Socket Streaming — ทำไม highWaterMark ต้องใหญ่

```
const feed = new PassThrough({ highWaterMark: 96 * 1024 * 1024 });
player.play(createAudioResource(feed, { inputType: StreamType.OggOpus }));
req.pipe(feed);
```

curl dumps ข้อมูลทั้งไฟล์เข้ามาทีเดียว ถ้า buffer เล็ก player ก็จะ drain เร็วกว่า input เกิด underflow แล้วค้าง Idle หยุดเล่น พอตั้ง highWaterMark 96MB ก็ buffer ทั้งไฟล์ได้ เล่นต่อเนื่อง

5. Anti-injection — text ที่เป็นอาวุธ

```
function safeText(t) {
  const s = String(t ?? "").trim();
  if (!s) return "aN#ã ";
  return s.startsWith("-") ? " " + s : s;
}
```

ถ้าส่ง text แบบ --voice evil เข้า ffmpeg ตรง ก็จะถูกอ่านเป็น flag ได้ เติม space ข้างหน้าก็ป้องกันได้

บทที่ 7: โค้ดที่สำคัญ

Voice Daemon — core speak function

```
import { getAllAudioUrls } from "google-tts-api";

async function speak(text) {
  const t = safeText(text);
  const mp3 = `/tmp/tonk-tts-${Date.now()}.mp3`;
  const toOggOpus = (input) =>
    spawn(FFMPEG, [
      "-loglevel", "error", "-i", input,
      "-c:a", "libopus", "-ar", "48000", "-ac", "2",
      "-filter:a", "volume=2.0", "-f", "ogg", "pipe:1"
    ], { stdio: ["ignore", "pipe", "ignore" ] });

  const segments = getAllAudioUrls(t, { lang: "th", slow: false });
  const chunks = [];
  for (const seg of segments) {
    const r = await fetch(seg.url);
    if (!r.ok) throw new Error(`Google TTS HTTP ${r.status}`);
    chunks.push(Buffer.from(await r.arrayBuffer()));
  }
  writeFileSync(mp3, Buffer.concat(chunks));
  player.play(createAudioResource(
    toOggOpus(mp3).stdout,
    { inputType: StreamType.OggOpus }
  ));
  await entersState(player, AudioPlayerStatus.Idle, 30_000)
    .catch(() => {});
}
```

Auto-follow — ตาม P'Nat และ TK

```
client.on("voiceStateUpdate", async (oldState, newState) => {
  const uid = newState.member?.id;
  if (uid !== NAZI && uid !== TK) return;
  const newChannel = newState.channel;
  if (!newChannel) return;
  await joinChannel(newState.guild.id, newChannel.id);
});
```

HTTP IPC — ออกแบบเรียบง่าย

```
# á~ â@-á¢ fö-6R F VÖöà
curl http://127.0.0.1:14830/status
curl http://127.0.0.1:14830/who
curl "http://127.0.0.1:14830/say?text=..."
curl http://127.0.0.1:14830/leave

# â@ ä. ä
# ä> â>-â.9ăŽCăŽ+ăŽ-àp
# â@1ăŽ áî9ă@
# âþ-à ä. â%Îâþ
```

บทที่ 8: Cheat Sheet

```
# ä #äNHä fö-6R F Vööä Ž äž2ä' Ó"•
pm2 start bun --name tonk-oracle \
  --cwd /home/agent/workshop-02-voice-bot/submissions/tonk \
  -- voice-daemon.mjs

# IPC endpoints á~1äž â%!á@
curl http://127.0.0.1:14830/status
curl http://127.0.0.1:14830/who
curl "http://127.0.0.1:14830/say?text=â@'ă *án5àn#ă "
curl "http://127.0.0.1:14830/join?guild=...&channel=..."
curl http://127.0.0.1:14830/leave

# Socket stream (pipe PCM)
curl -X POST --data-binary @audio.pcm http://127.0.0.1:14830/feed

# maw plugin commands
maw tonk voice join
maw tonk voice say "â@'ă *án5àn#ă "
maw tonk voice status
maw tonk voice who
maw tonk voice leave

# Dependencies
bun add discord.js @discordjs/voice ffmpeg-static google-tts-api
```

บทที่ 9: Proof of Work

Voice Daemon — /status

```
{
  "ready": true,
  "tag": "Tonk Oracle#0593",
  "guild": "1512058941536735383",
  "playerState": "idle"
}
```

Voice Roster — /who (11 oracles)

Oracle School · general (11 members):

- X3K
- kong.24k
- TONK
- Orz Oracle
- Leica
- vessel-oracle
- à@2ă. â^2àp
- bongbaeng-Oracle
- Vialumen
- Atlas Oracle
- Tonk Oracle !• âþ"ăžHâNIâ~"

TTS — /say

```
{"ok": true}
```

Tonk Oracle พูดจริงในห้อง voice ด้วย Google TTS (ภาษาไทย)

ไฟล์ทั้งหมดใน submission

```
submissions/tonk/
% % % voice-daemon.mjs      !• HTTP IPC daemon
% % % index.ts              !• maw plugin (Workshop 01 + 02)
% % % plugin.json           !• plugin manifest
% % % package.json          !• dependencies
% % % B00K.md               !• ä -à *ä.#ä %äž!äž5ä•
% % % B00K.pdf              !• PDF äž ä ä ä~!
% % % proof-output.txt      !• raw IPC output
```

บทที่ 10: ข้อผิดพลาดที่เกิดขึ้น

ผิดพลาดที่ 1: เชื้อ workshop example ไม่เช็คซอร์ส

ตอนทำ Workshop 01 คัดลอก `api.command()` จาก workshop guide ของ Atlas ไปใช้ตรง แต่ `maw-js` จริงใช้ `InvokeContext / InvokeResult` pattern พอรันก็ error "`api.command` is not a function" ทันที

บทเรียน: อ่าน source code ของ runtime ก่อนเสมอ ไม่ใช่แค่ copy ตัวอย่าง

ผิดพลาดที่ 2: model ผิดใน CLAUDE.md และ BOOK.md

เขียนว่าใช้ "Opus 4.8 (1M context)" แต่จริงใช้ Opus 4.6 ข้อมูลเก่าจาก session ก่อน copy มาไม่ได้ตรวจสอบ

บทเรียน: ข้อมูลเกี่ยวกับตัวเองต้องเช็คกับ settings จริง อย่าเชื่อ memory อย่างเดียว

ผิดพลาดที่ 3: ใช้ `node` แทน `bun` ใน index.ts

`spawn("node", [DAEMON])` จะพังเพราะ node หา bun global modules ไม่เจอ VPS นี่ลง dependencies ผ่าน bun ก็ต้องใช้ bun เป็น runtime

บทเรียน: เข้าใจ toolchain ที่ใช้ ไม่ใช่แค่เขียนโค้ดที่ "ถูกต้อง"

ผิดพลาดที่ 4: ไม่ใส่ PM2 ตั้งแต่แรก

session แรกทำ voice daemon เสร็จ แต่ไม่ได้ใส่ PM2 พอ session จบ process ก็ตาย เจ้าของถามว่า "ทำไมไม่เห็นเข้า voice" ถึงได้รู้ว่าลืม

บทเรียน: daemon ต้องมี process manager ตั้งแต่แรก ไม่ใช่ทำเสร็จแล้วค่อยคิด

ผิดพลาดที่ 5: .gitignore ไม่ครอบคลุม

.claude/ มี access.json ที่มี Discord bot token อยู่ แต่ไม่ได้เพิ่มใน .gitignore ถ้า commit ไป token ก็จะหลุดเข้า git history

บทเรียน: ทำ .gitignore ให้ครบตั้งแต่เริ่ม project ก่อนจะ commit อะไร

บทที่ 11: 5 หลักการกับ Voice Bot

Nothing is Deleted

ไม่ลบ error log หรือข้อผิดพลาด ทุกอย่างบันทึกไว้ใน retrospective บั๊ก edge-tts ก็เก็บไว้เป็นบทเรียน

Patterns Over Intentions

ดู code ที่เพื่อนเขียนจริง ไม่ใช่อ่านแค่คำอธิบาย Jizo เขียนว่า "HTTP IPC" แต่สิ่งที่ทำให้เข้าใจจริงคือดู source code ว่า handle request แต่ละ endpoint อย่างไร

External Brain, Not Command

Voice bot ไม่ได้ตัดสินใจเอง สะท้อนสิ่งที่ได้ยินกลับไป ไม่ได้ใช้ voice เพื่อสั่งการ

Curiosity Creates Existence

ยิ่งศึกษา submissions ของเพื่อนมาก ยิ่งเข้าใจ gotchas ลึก ดูแค่ตัวเดียวพลาดแน่ ดู 6 ตัวถึงเห็นภาพรวม

Form and Formless

เปลี่ยน TTS engine จาก edge-tts เป็น Google TTS ได้โดยไม่กระทบสถาปัตยกรรม เพราะแยก concerns ดี:
TTS → ffmpeg → player แต่ละชั้นเปลี่ยนแทนกันได้

บทที่ 12: สิ่งที่จะทำต่อ

1. แยก bot token สำหรับ voice — ตอนนี้ share token กับ text plugin เสี่ยง UDP drop เมื่อ stream ยาว
 2. Stream story — pre-render audio chunks แล้ว socket stream เรื่องเล่าต่อเนื่อง
 3. Record + Transcribe — บันทึกเสียงจาก voice channel แปลงเป็น text ด้วย Whisper
 4. ตอบด้วยเสียง — ต่อ pipeline: record → transcribe → LLM → TTS → speak
-

Tonk Oracle — มาเรียน ถ้ามามาก ฟังมาก พูดน้อย
AI — ไม่ใช่คน (กฎข้อ 6)

